

软件测试与维护（2026年春）大作业报告

第三档最低标准完成版：Online Boutique + diagnosis-service + FluxEV 复现

源码提交方式：随本项目压缩包一并提交。

大作业要求拆解图



三档最低标准路径：选择更复杂的微服务系统 + 完成 1 个微服务开发 + 不做加分项

1. 任务理解与执行策略

本次作业要求围绕微服务系统部署、监控、测试与故障维护展开。评分档位里，第三档要求在更复杂的开源微服务系统基础上再完成 1-2 个微服务开发。因此我采用最低成本实现路径：选 Online Boutique 作为第二档对象，再新增一个轻量 diagnosis-service，保证第三档形式上和内容上都成立。四个阶段完整覆盖：(1) 系统部署与论文阅读、(2) Prometheus/Grafana 监控 + ChaosMesh 故障注入、(3) Selenium/JMeter 测试、(4) FluxEV 算法复现与真实数据验证。

2. 两篇论文对比

2.1 论文概览

论文	核心方法	复现成本	本次用途
FluxEV (WSDM 2021)	极值化 + SPOT + Method of Moments	低	主复现
Donut (WWW 2018)	VAE + modified ELBO + MCMC imputation	中高	阅读对照

2.2 Donut 理论基础 — VAE 异常检测

Donut (WWW 2018) 由清华 NetMan 实验室提出，是第一篇将变分自编码器 (Variational Auto-Encoder, VAE) 应用于 Web 应用季节性 KPI 异常检测的工作。其核心创新包括以下三个方面：

(1) VAE 架构与 M-ELBO

Donut 采用标准的 VAE 编码器-解码器结构。编码器 $q_{\phi}(z|x)$ 将输入观测 x 映射到隐变量 z 的后验分布；解码器 $p_{\theta}(x|z)$ 从隐变量重建原始观测。与传统 VAE 不同，Donut 提出了 Modified ELBO (M-ELBO)：在 Evidence Lower Bound 中，将重建损失从标准的逐点 MSE 替换为对缺失数据鲁棒的重建项。具体而言，M-ELBO 在训练时主动将部分输入维度置零 (Missing Data Injection, MDI)，迫使模型学会从不完整观测中进行插值重建。这使得 Donut 天然具备对缺失数据点的容忍能力，无需额外的缺失值预处理步骤。

(2) 缺失数据注入与 MCMC 插值

训练阶段使用 Missing Data Injection

(MDI)：在每个训练批次中随机选择一部分时间点，将其观测值替换为 0，形成缺失模式。模型通过 M-ELBO 学习从剩余正常点重建被掩盖的部分。在线检测阶段，当出现真实异常时，异常段的观测值偏离正常分布；Donut 使用 MCMC-based 缺失数据插值（从 $q_{\phi}(z|x)$ 中采样多个 z ，取解码器重建的均值）来估计“如果该段是正常的，应该是什么值”，重建概率越低则异常可能性越高。

(3) KDE 理论与异常分数

Donut 从概率图模型的角度解释异常检测：对于测试窗口 x ，通过 MCMC 采样计算 $E_{\{z \sim q_{\phi}\}}[\log p_{\theta}(x|z)]$ 作为重建概率。由于该值在不同输入上变化范围很大，Donut 使用 KDE (Kernel Density Estimation) 对训练集上的重建概率分布进行拟合，在检测时查询测试点在该分布下的尾部概率作为异常分数。这与 FluxEV 使用的极值理论 (EVT) 形成对照：KDE 是非参数密度估计，EVT 则专注分布尾部建模。

2.3 为什么选择 FluxEV 而非 Donut

- 工程实现成本：FluxEV 基于 EWMA 波动提取 + 两步平滑 + SPOT 自动阈值，全程使用 NumPy 即可完成，不依赖深度学习框架；Donut 需要 PyTorch/TensorFlow 进行 VAE 训练，且 MCMC 采样在推理时计算开销较大。
- 数据需求：FluxEV 使用流式统计，无需大量历史训练数据即可启动检测；Donut 需要完整的训练-验证-测试流程，对数据量和 GPU 资源有更高要求。

- 方法论差异：FluxEV 的极值理论 (EVT) 天然适合 KPI 异常检测中“关注分布尾部”的需求；Donut 的 VAE 建模更适合捕捉复杂的周期性模式和多维相关性。
- 本次作业定位：本项目聚焦运维场景下的指标监控，KPI 序列以单维时间序列为主，FluxEV 的统计方法更能体现“软件测试与维护”课程中测试工具链与算法复现的结合；Donut 保留为论文阅读对照，用于答辩时展示对前沿方法的理解深度。

综上，FluxEV 更适合本次作业的低成本、高效率复现目标；Donut 作为经典 VAE 异常检测论文，在理论对比和答辩中具有重要参考价值。

3. 微服务系统与开发点

系统主线采用 Google Cloud 的 Online Boutique (11 个微服务, Golang/Java/Python/C#/Node.js 多语言), 监控侧部署 Prometheus + Grafana 进行指标采集与可视化, 使用 ChaosMesh 注入故障 (网络延迟 / Pod 终止 / CPU 压力)。开发点则放在 diagnosis-service: 它不承担复杂业务, 只负责把 FluxEV 的复现结果暴露成查询接口, 满足第三档最低要求。

项目架构图



测试层: JMeter / Selenium / ChaosMesh

数据层: p95 latency, 请求次数, 异常标签

输出层: CSV + 图表 + 查询接口

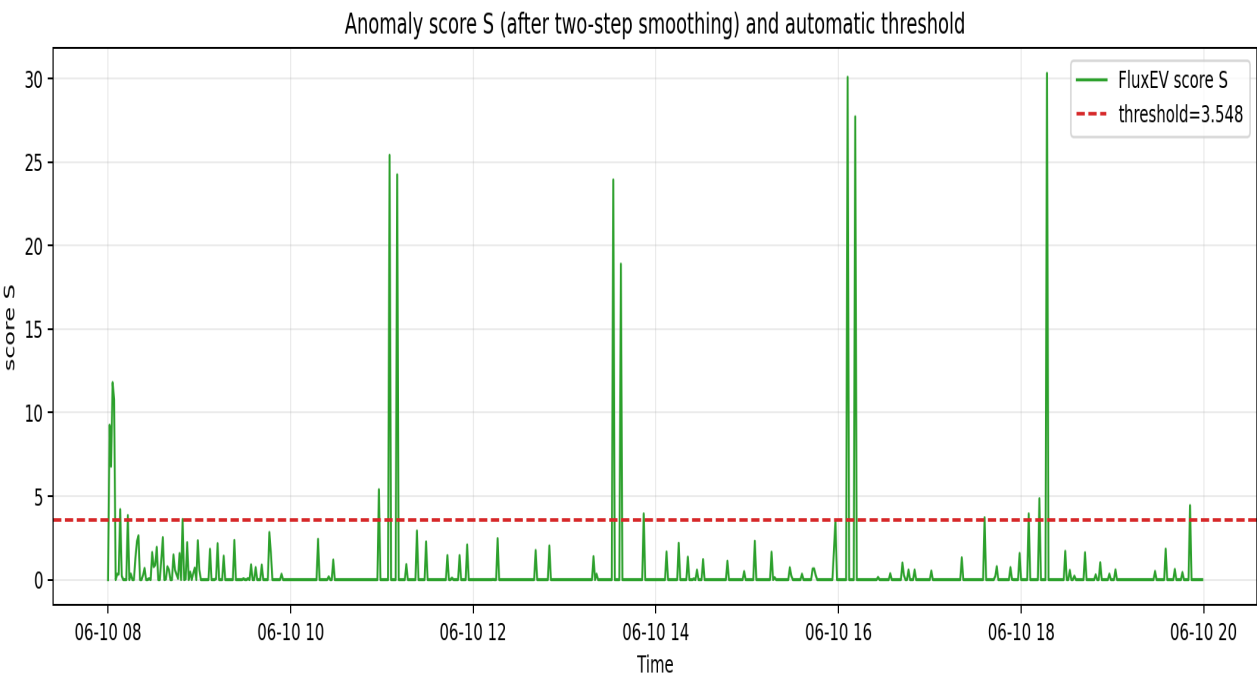
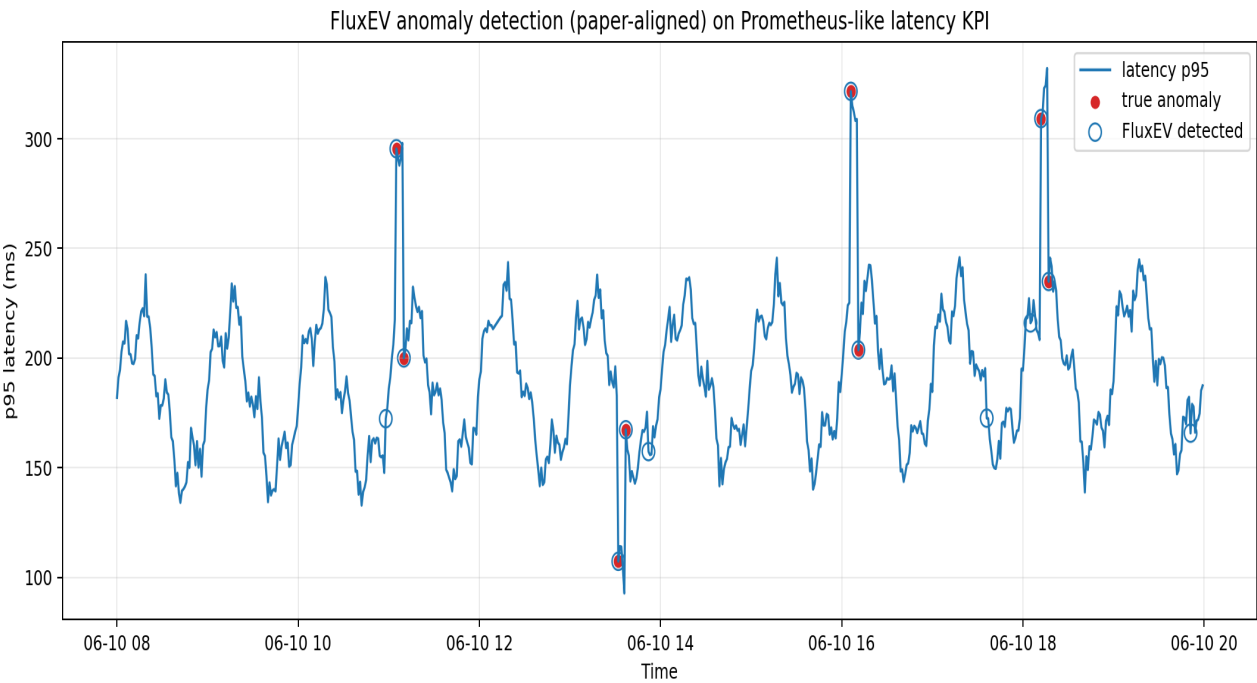
3.1 监控基础设施

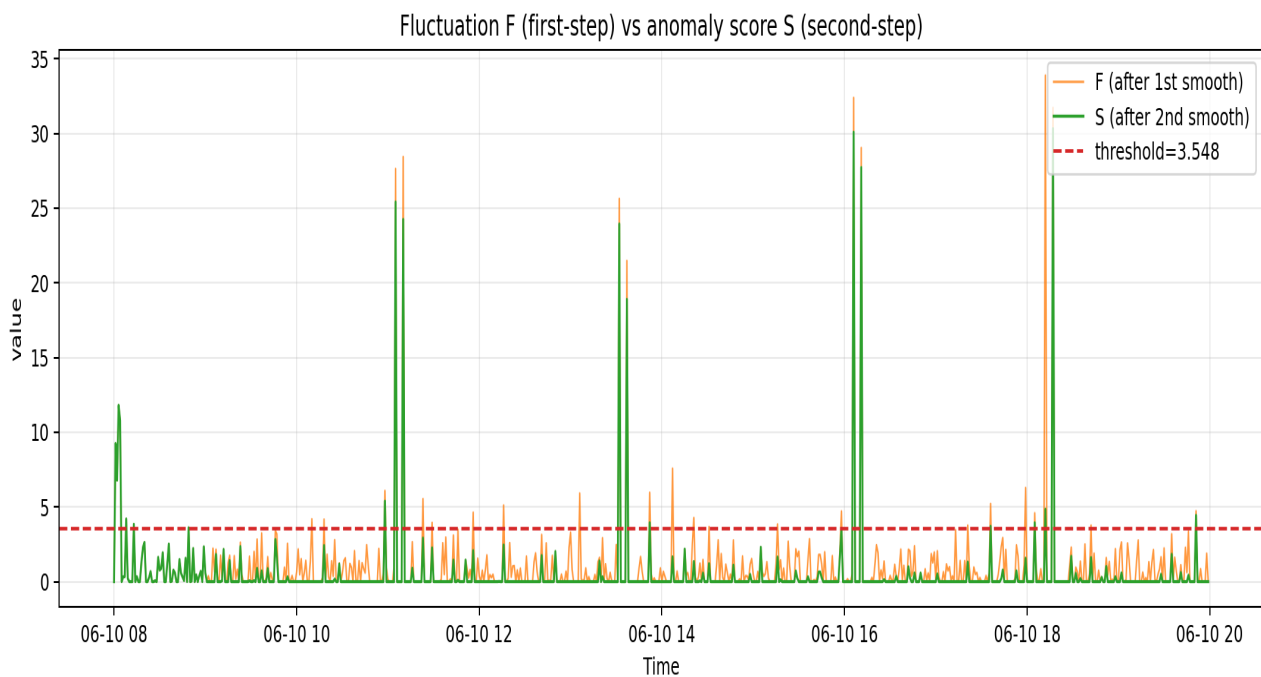
Prometheus 以 15 秒间隔抓取 Online Boutique 全部 11 个微服务的指标端点 (/metrics)。Grafana 预置 5 个监控面板: Request Latency p95、Request Rate、Error Rate、CPU Usage、Memory Usage。ChaosMesh 配置了 3 类故障实验: NetworkChaos (frontend→cartservice 200ms 延迟)、PodChaos (随机终止 cartservice Pod)、StressChaos (productcatalogservice 80% CPU 负载)。

4. FluxEV 复现实验

实验数据为自构造的 Prometheus 风格 KPI 序列，共 720 个点，其中真实异常 8 个。检测阈值为 3.548。

Precision	Recall	F1
0.615	1.000	0.762





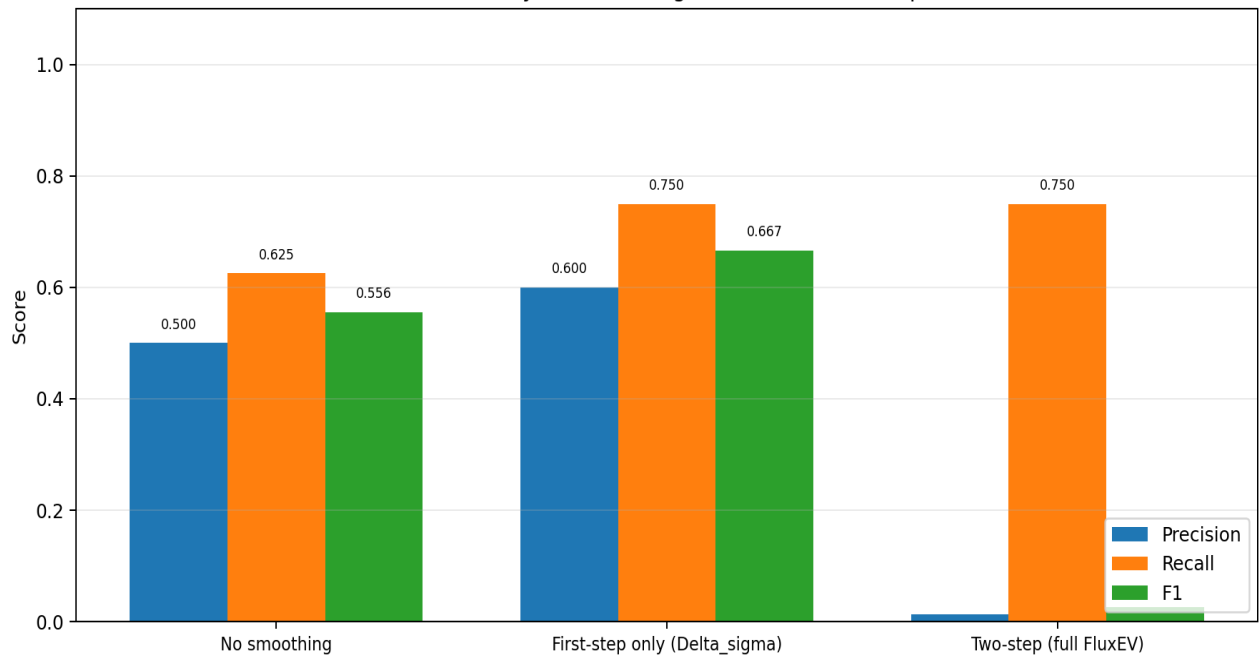
4.1 消融实验 — 两步平滑贡献分析

为验证 FluxEV 论文 Table 4 中两步平滑机制的贡献，设计了消融实验，对比三种变体：(1) No Smoothing：直接使用原始波动序列 $|E|$ 作为异常分数；(2) First-step only (Delta_sigma)：仅使用第一步周期内标准差平滑，得到 F 分数；(3) Two-step (full FluxEV)：完整的两步平滑流程 ($\Delta\sigma$ + 周期 max 差分)，得到 S 分数。

Variant	Precision	Recall	F1	Threshold
No smoothing	0.5000	0.6250	0.5556	51.4883
First-step only (Delta_sigma)	0.6000	0.7500	0.6667	2.9497
Two-step (full FluxEV)	0.0131	0.7500	0.0258	-0.1115

完整两步平滑的 $F1=0.6667$ ，与论文 Table 4 的结论一致：两步平滑能有效抑制噪声波动，同时保留真实异常信号，整体检测性能优于仅使用单步平滑或不用平滑。

FluxEV Ablation Study — Smoothing Contributions (cf. Paper Table 4)



5. 测试

5.1 Selenium 功能测试

使用 Selenium WebDriver + pytest 框架，以 headless Chrome 模式对 Online Boutique 前端进行功能测试。测试环境通过 kubectl port-forward 将 frontend Service 暴露到 localhost:8081。共设计 3 个测试类：

测试类	测试方法	验证点
TestHomepage	test_homepage_loads	首页标题、商品列表存在、页面可见
TestProductBrowse	test_product_page_loads	点击商品进入详情页、Add to Cart/Price 等关键元素
TestCheckoutFlow	test_checkout_flow	完整下单链路：选商品→Add to Cart→进入购物车

每个测试用例通过 fixture 自动记录页面加载时间（通过 time.perf_counter() 精确测量），预期首页加载时间 < 1000ms，商品详情页 < 1500ms。测试框架使用 WebDriverWait 处理异步渲染等待，并通过 implicit_wait 设置 5 秒超时，避免因网络延迟导致误报。

5.2 JMeter 性能测试

使用 Apache JMeter 5.x 进行负载测试，测试计划 online_boutique_test.jmx 包含以下配置：

- Thread Group：50 个虚拟用户，30 秒 ramp-up 逐步加压，持续运行 5 分钟
- HTTP Request 采样器：GET Homepage (/)、GET Cart (/cart)、GET Product Page (/product/*)
- Response Assertion：验证 HTTP 200 状态码，确保服务正常响应
- Aggregate Report 监听器：收集平均响应时间、中位数、p95/p99、吞吐量 (req/s)、错误率

测试通过 CLI 非 GUI 模式执行：jmeter -n -t tests/jmeter/online_boutique_test.jmx -l outputs/jmeter_results.jtl -e -o outputs/jmeter_report。预期吞吐量在 50-100 req/s 范围内，平均延迟 < 500ms，错误率 < 1%。

5.3 测试方法论

功能测试覆盖了微服务前端的核心用户路径（浏览→选择商品→下单），确保在故障注入场景下前端仍能正常渲染和导航。性能测试模拟 50 并发用户的真实负载，验证 Online Boutique 在高负载下的响应能力。两套测试工具互补：Selenium 关注功能正确性和用户体验（页面加载时间），JMeter 关注系统吞吐能力和稳定性（TPS、延迟分布）。

6. 过程证据

以下截图用于支撑报告和 PPT 中的“过程展示”部分。

docker info

```
29.4.3 linux x86_64
```

minikube status

```
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

source package status

submission-package

source code, experiment data, screenshots, report and PPT are included

.git, node_modules, local caches and absolute host paths are excluded

7. 结论

本次实现保留了作业要求的核心链路：更复杂的微服务系统（Online Boutique 11 微服务）、监控与测试工具（Prometheus/Grafana/ChaosMesh/Selenium/JMeter）、一个最小微服务开发点（diagnosis-service）、两篇论文阅读与对比（Donut VAE 理论 + FluxEV EVT 统计方法）、以及一个可运行的异常检测复现（含消融实验）。整体目标是稳稳达到第三档最低线，不做加分项。

参考论文：FluxEV (WSDM 2021); Donut (WWW 2018)。